

```

/**
 * server/lib/db-boot-ready.ts
 *
 * Fix #3 – Boot DB Queries Fire Before Pool Is Warm
 *
 * PROBLEM: Every session shows boot-sequence queries failing immediately:
 *   [Boot] Full queue reset skipped: Failed query: UPDATE autopilot_queue...
 *   [Boot] BF6 platform_shorts pull skipped: Failed query: WITH ranked AS...
 *   [Boot] runVodOptimizationCycle failed: Failed query: select distinct...
 *
 * These fail because Neon PostgreSQL connection pool isn't fully warmed
 * when the boot sequence fires (Express starts, then immediately runs
 * state-mutating queries before the first connection succeeds).
 *
 * SOLUTION: A boot-ready guard that pings the DB with exponential backoff
 * before any boot queries run. All boot-sequence code should be wrapped in
 * awaitDbReady() before executing.
 *
 * Usage:
 *   import { awaitDbReady } from '../lib/db-boot-ready.js';
 *
 *   // In server/index.ts boot sequence:
 *   await awaitDbReady();
 *   await runShortsScheduleReset();
 *   await runQueueReset();
 *   await runBF6PlatformShortsPull();
 *   // etc.
 */

import { db }          from '../db.js';
import { sql }          from 'drizzle-orm';
import { createLogger } from './logger.js';

const log = createLogger('db-boot-ready');

const MAX_ATTEMPTS      = 10;
const BASE_DELAY_MS     = 500;
const MAX_DELAY_MS      = 10_000;

let isReady = false;

/**
 * Waits for the DB to be responsive before returning.
 * Uses exponential backoff. Resolves once a SELECT 1 succeeds.
 * Safe to call multiple times – resolves immediately if already ready.
 */
export async function awaitDbReady(): Promise<void> {
  if (isReady) return;

```

```

log.info('[DbBootReady] Waiting for DB connection pool to warm...');

for (let attempt = 1; attempt <= MAX_ATTEMPTS; attempt++) {
  try {
    await db.execute(sql`SELECT 1`);
    isReady = true;
    log.info('[DbBootReady] DB ready after ${attempt} attempt(s)`);
    return;
  } catch (err: any) {
    const delay = Math.min(BASE_DELAY_MS * Math.pow(2, attempt - 1), MAX_DELAY_MS);
    log.warn(
      `[DbBootReady] DB not ready (attempt ${attempt}/${MAX_ATTEMPTS}): ` +
      `${err.message?.slice(0, 80)} - retrying in ${delay}ms`
    );
    await new Promise(r => setTimeout(r, delay));
  }
}

// If we get here, DB is still not responding – log critical and continue
// rather than hanging the server forever. Boot queries will still fail
// but the server will at least start.
log.error(
  `[DbBootReady] DB did not respond after ${MAX_ATTEMPTS} attempts. ` +
  'Boot queries may fail. Check Neon connection string and pool settings.'
);
}

/**
 * Wrap a boot-sequence function so it only runs after DB is ready,
 * and logs skips instead of throwing.
 *
 * Usage:
 *   await runWhenReady('Shorts schedule reset', runShortsScheduleReset);
 *   await runWhenReady('BF6 shorts pull',      runBF6PlatformShortsPull);
 *   await runWhenReady('Queue reset',          runQueueReset);
 */
export async function runWhenReady(
  label: string,
  fn: () => Promise<void>,
): Promise<void> {
  await awaitDbReady();
  try {
    await fn();
    log.info('[Boot] ${label} completed`);
  } catch (err: any) {
    log.warn('[Boot] ${label} skipped:`, { error: err.message?.slice(0, 200) });
  }
}

```

